< INTERVIEW PREP GUIDE >

PYTHON

Important Interview Questions

Definitions | Examples | Coding Questions

1. Python Basics

Q1. What is Python?

Ans: Python is a high-level, interpreted, and easy-to-learn programming language.

Q2. What are the main features of Python?

Ans: Python is easy to learn, readable, interpreted, dynamically typed, and supports object-oriented programming.

Q3. What is a variable in Python?

Ans: A variable is a name used to store a value.

Q4. What are comments in Python?

Ans: Comments are notes in code that Python does not execute.

Q5. What is indentation in Python?

Ans: Indentation means spaces at the beginning of a line. Python uses indentation to define code blocks.

2. Data Types and Type Conversion

Q6. What is a data type?

Ans: A data type defines the kind of value stored in a variable.

Q7. What are the main built-in data types in Python?

Ans: `int`, `float`, `complex`, `str`, `bool`, `list`, `tuple`, `set`, `dict`, `NoneType`.

Q8. What is type conversion?

Ans: Type conversion means changing a value from one data type to another.

Q9. What is the difference between implicit and explicit type conversion?

- **Implicit:** Python converts a value automatically.
- **Explicit:** The programmer converts a value using functions such as `int()`, `float()`, and `str()`.

3. Operators

Q10. What are operators in Python?

Ans: Operators are symbols or keywords used to perform operations on values and variables.

Q11. What are the main types of operators?

Ans: Arithmetic, comparison, assignment, logical, bitwise, membership, and identity operators.

Q12. What are arithmetic operators?

Ans: They perform mathematical operations, such as `+`, `-`, `*`, `/`, `//`, `%`, `**`.

Q13. What are comparison operators?

Ans: They compare two values and return `True` or `False`. Examples: `==`, `!=`, `>`, `<`, `>=`, `<=`.

Q14. What are logical operators?

Ans: Logical operators combine or reverse conditions. They are `and`, `or`, `not`.

Q15. What are membership operators?

Ans: `in` and `not in` check whether a value exists in a collection or sequence.

4. Collection Data Types

Q16. What is a collection data type?

Ans: A collection data type stores multiple values in one object.

Q17. What is a list?

Ans: A list is an ordered, mutable collection that allows duplicate values.

```
numbers = [10, 20, 30, 10]
```

Q18. What is a tuple?

Ans: A tuple is an ordered, immutable collection that allows duplicate values.

```
numbers = (10, 20, 30)
```

Q19. What is a set?

Ans: A set is a collection of unique elements. It does not support indexing.

```
numbers = {10, 20, 30}
```

Q20. What is a dictionary?

Ans: A dictionary stores data in key-value pairs.

```
student = {"name": "Jayesh", "age": 22}
```

Q21. What is the difference between a list and a tuple?

Ans: A list is mutable, while a tuple is immutable.

Q22. What is the difference between a list and a set?

Ans: A list maintains order and allows duplicates. A set stores unique elements and does not guarantee positional order.

Q23. What is the difference between a list and a dictionary?

Ans: A list stores individual items, while a dictionary stores key-value pairs.

Q24. What is mutable and immutable?

- **Mutable:** An object that can be changed after creation, such as a list or dictionary.
- **Immutable:** An object that cannot be changed after creation, such as an integer, string, or tuple.

Q25. What is indexing?

Ans: Indexing means accessing an individual element by its position.

```
name = "Python"
print(name[0])   # P
print(name[-1])  # n
```

Q26. What is slicing?

Ans: Slicing means accessing a part of a sequence.
Syntax: `sequence[start:stop:step]`

Q27. What is a nested list?

Ans: A nested list is a list containing another list.

5. List, Set and Dictionary Comprehensions

Q28. What is list comprehension?

Ans: List comprehension is a short way to create a list.

```
squares = [x**2 for x in range(1, 6)]
print(squares)
# [1, 4, 9, 16, 25]
```

Q29. What is set comprehension?

Ans: Set comprehension is a short way to create a set.

```
squares = {x**2 for x in range(1, 6)}
```

Q30. What is dictionary comprehension?

Ans: Dictionary comprehension is a short way to create a dictionary.

```
squares = {x: x**2 for x in range(1, 6)}
```

6. Built-in Functions

Q31. What is a built-in function?

Ans: A built-in function is a function already provided by Python.

Q32. Name some important built-in functions.

Ans: `print()`, `input()`, `len()`, `type()`, `range()`, `sum()`, `min()`, `max()`, `sorted()`, `abs()`.

Q33. What is the len() function?

Ans: `len()` returns the number of items in an object.

```
print(len("Python"))  # 6
```

Q34. What is the range() function?

Ans: `range()` generates a sequence of integers, commonly used with loops.

```
print(list(range(1, 5)))
# [1, 2, 3, 4]
```

Q35. What is the difference between print() and return?

Ans: `print()` displays a value, while `return` sends a value back from a function.

Q36. What is the input() function?

Ans: `input()` accepts user input and returns it as a string by default.

7. Functions

Q37. What is a function?

Ans: A function is a reusable block of code that performs a specific task.

Q38. What are the types of functions in Python?

Ans: Two basic categories are built-in functions and user-defined functions. Python also supports anonymous lambda functions.

Q39. What is a user-defined function?

Ans: A user-defined function is a function created by a programmer using the `def` keyword.

```
def greet():
    print("Hello")
```

Q40. What are parameters and arguments?

Ans: Parameters are variables written in a function definition. Arguments are the actual values passed when calling the function.

Q41. What is a default argument?

Ans: A default argument is a parameter with a value that is used when no argument is provided for it.

```
def greet(name="Jayesh"):
    print(name)

greet()
```

Q42. What is a lambda function?

Ans: A lambda function is a small anonymous function written using the `lambda` keyword.

```
square = lambda x: x * x
print(square(5))  # 25
```

8. Flow Control and Conditional Statements

Q43. What is flow control in Python?

Ans: Flow control determines the order in which statements execute in a program.

Q44. What are the main flow control statements?

Ans: Conditional statements, loops, and jump statements such as `break`, `continue`, and `pass`.

Q45. What is a conditional statement?

Ans: A conditional statement executes code based on whether a condition is true or false.

Q46. What is an if statement?

Ans: An `if` statement executes a block when its condition is true.

```
age = 20
if age >= 18:
    print("Eligible")
```

Q47. What is an if-else statement?

Ans: It selects between two blocks of code based on a condition.

Q48. What is an if-elif-else statement?

Ans: It checks multiple conditions and executes the first matching block.

```
marks = 75
if marks >= 80:
    print("A")
elif marks >= 60:
    print("B")
else:
    print("C")
```

Q49. What is a nested if statement?

Ans: A nested `if` is an `if` statement inside another `if` statement.

Q50. What is a loop in Python?

Ans: A loop repeatedly executes a block of code.

Q51. What are the types of loops in Python?

Ans: Python has two main loop statements: `for` and `while`.

Q52. What is a for loop?

Ans: A `for` loop iterates over the items of an iterable, such as a list, string, or range.

```
for i in range(1, 6):
    print(i)
```

Q53. What is a while loop?

Ans: A `while` loop repeats while its condition remains true.

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

Q54. What is the difference between a for loop and a while loop?

Ans: A `for` loop is commonly used to iterate over a sequence or iterable. A `while` loop is commonly used when repetition depends on a condition.

Q55. What is a nested loop?

Ans: A nested loop is a loop inside another loop.

```
for i in range(2):
    for j in range(3):
        print(i, j)
```

Q56. What is the break statement?

Ans: `break` immediately terminates the nearest enclosing loop.

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
# 1 2
```

Q57. What is the continue statement?

Ans: `continue` skips the rest of the current iteration and moves to the next iteration.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
# 1 2 4 5
```

Q58. What is an infinite loop?

Ans: An infinite loop continues indefinitely because its stopping condition is never reached or its condition always remains true.

```
while True:
    print("Running forever")
```

9. Quick Revision - One-Line Definitions

1. Data Types	Define the type of value stored in a variable, such as numbers, text, or collections.
2. Type Conversion	Changing a value from one data type to another.
3. Python Flow Control	Determines the order in which statements are executed in a program.
4. Python Operators	Symbols or keywords used to perform operations on values and variables.
5. Collection Data Types	Store multiple values in a single variable or object.
6. List	An ordered, mutable collection that allows duplicate values.
7. Tuple	An ordered, immutable collection that allows duplicate values.
8. Set	A collection of unique elements without guaranteed positional order.
9. Comprehensions	A short and simple way to create collections from existing iterables.
10. Built-in Functions	Functions already provided by Python that we can use directly.
11. User-Defined Functions	Functions created by programmers to perform specific tasks.
12. Conditional Statements	Allow a program to make decisions based on conditions.
13. Lambda Functions	A small anonymous function created using the lambda keyword.
14. Loop Statements	Used to execute a block of code repeatedly.

10. Important Python Coding Questions

Try solving these programs without looking at the answers.

1. Print numbers from 1 to 10.

SOLUTION

```
for i in range(1, 11):
    print(i)
```

OUTPUT

```
1 2 3 4 5 6 7 8 9 10  (one per line)
```

2. Print even numbers from 1 to 20.

SOLUTION

```
for i in range(2, 21, 2):
    print(i)
```

OUTPUT

```
2 4 6 8 10 12 14 16 18 20  (one per line)
```

3. Find the sum of numbers from 1 to 10.

SOLUTION

```
total = 0
for i in range(1, 11):
    total += i
print(total)
```

OUTPUT

```
55
```

4. Print a multiplication table of 5.

SOLUTION

```
for i in range(1, 11):
    print(5, 'x', i, '=', 5 * i)
```

OUTPUT

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

5. Find the factorial of 5.

SOLUTION

```
factorial = 1
for i in range(1, 6):
    factorial *= i
print(factorial)
```

OUTPUT

```
120
```

6. Reverse a string using a loop.

SOLUTION

```
text = 'python'
reversed_text = ''
for char in text:
    reversed_text = char + reversed_text
print(reversed_text)
```

OUTPUT

```
nohtyp
```

7. Count vowels in a string.

SOLUTION

```
text = 'python programming'
count = 0
for char in text.lower():
    if char in 'aeiou':
        count += 1
print(count)
```

OUTPUT

```
4
```

8. Print this star pattern.

SOLUTION

```
for i in range(1, 5):
    print('*' * i)
```

OUTPUT

```
*
**
***
****
```

9. Find the largest number in a list without max().

SOLUTION

```
numbers = [10, 50, 20, 40]
largest = numbers[0]
for number in numbers:
    if number > largest:
        largest = number
print(largest)
```

OUTPUT

```
50
```

10. Check whether a number is prime.

SOLUTION

```
n = 7
is_prime = n > 1
for i in range(2, n):
    if n % i == 0:
        is_prime = False
        break
print(is_prime)
```

OUTPUT

```
True
```

Best of luck for your interview!